Submission 1

2026-10-21

Step 1: Clean the covariate data

```
head(GSE.covar.raw)
```

```
## # A tibble: 6 x 10
##   subjectID geoID      mat_age_yrs        preg_preeclampsia delivery_gestAge_wks
##   <chr>     <chr>      <chr>              <chr>             <chr>
## 1 6053887-M GSM2264346 age_years: 31.066~ pregnancy_condit~ gestation_weeks: 37~
## 2 3064425-M GSM2264347 age_years: 27.794~ pregnancy_condit~ gestation_weeks: 39~
## 3 6094898-M GSM2264348 age_years: 34.318~ pregnancy_condit~ gestation_weeks: 38~
## 4 7050617-M GSM2264349 age_years: 33.426~ pregnancy_condit~ gestation_weeks: 39~
## 5 7016015-M GSM2264350 age_years: 28.279~ pregnancy_condit~ gestation_weeks: 40~
## 6 7054360-M GSM2264351 age_years: 23.501~ pregnancy_condit~ gestation_weeks: 41~
## # i 5 more variables: sampleCollection_gestAge_wks <chr>,
## #   batch_Microarray <chr>, mat_BMI <chr>, mat_vitD <chr>, mat_asthma <chr>
```

```
summary(GSE.covar.raw)
```

```
##      subjectID         geoID        mat_age_yrs  preg_preeclampsia
##  Length   :157   Length   :157   Length   :157   Length   :157
##  N.unique :157   N.unique :157   N.unique :157   N.unique :  2
##  N.blank  :  0   N.blank  :  0   N.blank  :  0   N.blank  :  0
##  Min.nchar:  9   Min.nchar: 10   Min.nchar: 13   Min.nchar: 27
##  Max.nchar:  9   Max.nchar: 10   Max.nchar: 22   Max.nchar: 33
##  delivery_gestAge_wks sampleCollection_gestAge_wks  batch_Microarray
##  Length   :157        Length   :157                 Length   :157
##  N.unique : 44        N.unique : 53                 N.unique :  8
##  N.blank  :  0        N.blank  :  0                 N.blank  :  0
##  Min.nchar: 19        Min.nchar: 32                 Min.nchar: 41
##  Max.nchar: 28        Max.nchar: 41                 Max.nchar: 41
##       mat_BMI         mat_vitD       mat_asthma
##  Length   :157   Length   :157   Length   :157
##  N.unique :140   N.unique :132   N.unique :  2
##  N.blank  :  0   N.blank  :  0   N.blank  :  0
##  Min.nchar: 20   Min.nchar: 45   Min.nchar: 31
##  Max.nchar: 22   Max.nchar: 48   Max.nchar: 31
```

```
# Data Wrangling time! Whoohoo! Seems like there are some extra text in some
# columns. With the help of the trusty mutate function
# we can clean these cells up!
GSE.covar <- GSE.covar.raw %>%
```

```
  mutate(
    mat_age_yrs = str_remove_all(mat_age_yrs,"age_years: "),
    preg_preeclampsia = str_remove_all(preg_preeclampsia, "pregnancy_condition: "),
    delivery_gestAge_wks = str_remove_all(delivery_gestAge_wks, "gestation_weeks: "),
    sampleCollection_gestAge_wks = str_remove_all(sampleCollection_gestAge_wks, "blood_draw_at_gestatio
    batch_Microarray = str_remove_all(batch_Microarray, "microarray_hybridization_date: "),
    mat_BMI = str_remove_all(mat_BMI, "bmi_kg/m2: "),
    mat_vitD = str_remove_all(mat_vitD, "vitamind_25\\(0h\\)_ng/ml_whole_blood_baseline: "),
    mat_asthma = ifelse(str_ends(mat_asthma, "1"), "1", "0"))


# I also noticed all variables are stored as characters, for safe keeping
# I am going to change the ones that look like numeric measure to numeric
GSE.covar <- GSE.covar %>%
  mutate(across(c(mat_age_yrs, delivery_gestAge_wks, sampleCollection_gestAge_wks,
                  mat_BMI, mat_vitD),
                as.numeric))


head(GSE.covar)
```

```
## # A tibble: 6 x 10
##   subjectID geoID      mat_age_yrs preg_preeclampsia delivery_gestAge_wks
##   <chr>     <chr>            <dbl> <chr>                            <dbl>
## 1 6053887-M GSM2264346        31.1 Preeclampsia                      37.4
## 2 3064425-M GSM2264347        27.8 Normal                            39.4
## 3 6094898-M GSM2264348        34.3 Normal                            38.9
## 4 7050617-M GSM2264349        33.4 Normal                            39.1
## 5 7016015-M GSM2264350        28.3 Preeclampsia                      40.1
## 6 7054360-M GSM2264351        23.5 Normal                            41.1
## # i 5 more variables: sampleCollection_gestAge_wks <dbl>,
## #   batch_Microarray <chr>, mat_BMI <dbl>, mat_vitD <dbl>, mat_asthma <chr>
```

```
dim(GSE.covar)
```

```
## [1] 157  10
```

Step 2: Make the categorical variables

```
# Categorical variable #1: preclampsia status (already categories, so we just
# tell R it's a factor and put "Normal" first so it is the reference group).
# Categorical variable #2: vitamin D status, DERIVED from the continuous
# mat_vitD column. Under 20 ng/mL is a commonly used cutoff for deficiency.

GSE.covar <- GSE.covar %>%
  mutate(
    preg_preeclampsia = factor(preg_preeclampsia,
                               levels = c("Normal", "Preeclampsia")),
    vitD_status = factor(ifelse(mat_vitD < 20,
                                "Deficient (<20 ng/mL)",
```

```
                                "Sufficient (>=20 ng/mL)"),
                         levels = c("Deficient (<20 ng/mL)",
                                    "Sufficient (>=20 ng/mL)")))


# Check that the new variable makes sense: group sizes, and that the cutoff
# really split the vitamin D values where we said it would.
table(GSE.covar$preg_preeclampsia, GSE.covar$vitD_status)
```

```
##
##                 Deficient (<20 ng/mL) Sufficient (>=20 ng/mL)
##   Normal                           52                      58
##   Preeclampsia                     26                      21
```

```
GSE.covar %>% group_by(vitD_status) %>% summarise(min = min(mat_vitD), max = max(mat_vitD))
```

```
## # A tibble: 2 x 3
##   vitD_status               min   max
##   <fct>                   <dbl> <dbl>
## 1 Deficient (<20 ng/mL)    5.11  19.9
## 2 Sufficient (>=20 ng/mL) 20.4   80.8
```

Step 3: Clean the gene expression data and attach gene names

```
# The first column of the expression file has a blank header, so we name it
# PROBEID. Probe IDs are just labels, so we store them as text in both tables
# so that R can match them up.

GSE.GE <- GSE.GE.raw %>%
  rename(PROBEID = 1) %>%
  mutate(PROBEID = as.character(PROBEID))

GSE.GN <- GSE.GN.raw %>%
  mutate(PROBEID = as.character(PROBEID))

# Step A: look up the gene name for each probe
# Step B: pivot_longer turns the sideways table (one column per person) into a
#         tall table with one row per gene per person.

GSE.GE <- GSE.GE %>%
  left_join(GSE.GN, by = "PROBEID") %>%
  pivot_longer(cols = -c(PROBEID, GENE),
               names_to = "geoID",
               values_to = "expression")



# DO NOT INCLUDE STOPIFNOT

# Safety checks: every probe got a gene name, and the sample IDs in the
```

```r
# expression data are exactly the sample IDs in the covariate data.
stopifnot(!any(is.na(GSE.GE$GENE)))
stopifnot(setequal(unique(GSE.GE$geoID), GSE.covar$geoID))

head(GSE.GE)
```

```
## # A tibble: 6 x 4
##   PROBEID GENE    geoID      expression
##   <chr>   <chr>   <chr>           <dbl>
## 1 8089659 CCDC191 GSM2264363       6.18
## 2 8089659 CCDC191 GSM2264379       6.11
## 3 8089659 CCDC191 GSM2264394       6.32
## 4 8089659 CCDC191 GSM2264418       5.91
## 5 8089659 CCDC191 GSM2264449       6.35
## 6 8089659 CCDC191 GSM2264452       5.91
```

Step 4: Pick our genes and build the plotting table

```r
#DO NOT USE STOPIFNOT
# Not every gene in the linker file is in the expression data, so check first!
genes_of_interest <- c("RUNDC3A", "LYN", "IGFBP1")
stopifnot(all(genes_of_interest %in% GSE.GE$GENE))   # stops w an error if a gene is missing

# Each gene should have exactly 157 rows (one probe, one row per person)
GSE.GE %>% filter(GENE %in% genes_of_interest) %>% count(GENE)
```

```
## # A tibble: 3 x 2
##   GENE        n
##   <chr>   <int>
## 1 IGFBP1    157
## 2 LYN       157
## 3 RUNDC3A   157
```

```r
# Keep just our genes, then attach each person's covariates using geoID
dat_all <- GSE.GE %>%
  filter(GENE %in% genes_of_interest) %>%
  left_join(GSE.covar, by = "geoID")

# Should be 157 rows: one per person
stopifnot(nrow(dat_all) == length(genes_of_interest) * nrow(GSE.covar))
```

Step 5: Build the plotting function

```r
# Inputs:
#   data = long data frame with one row per person per gene. It needs the
#   column GENE, expression, geoID, mat_BMI, preg_Preeclampsia, and vitD_status

#   gene = character vector with the names of 1 or more genes
```

```r
# Output: prints 3 plots per gene, and (invisibly) returns the plots plus a
# small summary table of the numbers shown on them

plot_gene_expression <- function(data, genes) {
  plots <- list()       # holds every plot so we can reuse them later
  gene_stats <- list() # holds one row of summary numbers per gene

  # Loop through each gene and make its 3 plots
  for (g in genes){

    # Keep only this gene's rows
    d <- data %>% filter(GENE == g)


# PLOT 1: Histogram of gene expression
    m <- mean(d$expression); s <- sd(d$expression)

    p1 <- ggplot(d, aes(x = expression)) +
      geom_histogram(bins = 20, fill = "dodgerblue", color = "white") +
      geom_vline(xintercept = m, color = "orange", linewidth = 1, linetype = "dashed") +
      annotate("text", x = Inf, y = Inf, hjust = 1.05, vjust = 1.5, size = 5,
               label = paste0("n = ", nrow(d),
                              "\nMean = ", round(m,2),
                              "\nSD = ", round(s, 2))) +
      scale_y_continuous(expand = expansion(mult = c(0, 0.4))) +
      labs(x = paste(g, "expression level"),
           y = "Number of samples") +
      theme_classic(base_size = 14)

# PLOT 2: Scatterplot of BMI vs. Gener expression

    # Pearson correlation between BMI and expression for this gene
    ct <- cor.test (d$mat_BMI, d$expression)

    # Write very small p-values as "p < 0.001" so they don't up as "p = 0.00"
    p_text <- ifelse(ct$p.value < 0.001, "p < 0.001", sprintf("p = %.2f", ct$p.value))


    p2 <- ggplot(d, aes(x = mat_BMI, y = expression)) +
      geom_point(alpha = 0.7, size = 2, color = "dodgerblue") +
      geom_smooth(method = "lm", formula = y~ x, color = "orange",
                  fill = "orange", alpha = 0.2) +
     annotate("text", x = Inf, y = Inf, hjust = 1.1, vjust = 1.5, size = 5,
         label = paste0("r = ", round(ct$estimate, 2),
                        "\n", p_text,
                        "\nn = ", sum(!is.na(d$mat_BMI)))) +
      scale_y_continuous(expand = expansion(mult = c(0.05, 0.4))) +
      labs(x = expression("Maternal BMI (kg/m"^2*")"),
           y = paste(g, "expression level")) +
      theme_classic(base_size = 14)


## PLOT 3: Boxplot of expression by both categorical variables
```

```
    # Group sizes, with the height for the "n = ..." labels set just above this
    # gene's highest point (the old fixed height of 11.4 only worked for one gene)
    n_labels <- d %>%
      count(preg_preeclampsia, vitD_status) %>%
      mutate(y = max(d$expression) + 0.05 * diff(range(d$expression)))


     p3 <- ggplot(d, aes(x = preg_preeclampsia, y = expression, fill = vitD_status)) +
      geom_boxplot(outlier.shape = NA, alpha = 0.7,
                   position = position_dodge(width = 0.8), width = 0.7) +
      geom_point(aes(group = vitD_status),
                 position = position_jitterdodge(jitter.width = 0.15,
                                                 dodge.width = 0.8, seed = 1),
                 alpha = 0.5, size = 1.5, show.legend = FALSE) +
      geom_text(data = n_labels, aes(y = y, label = paste0("n = ", n)),
                position = position_dodge(width = 0.8), size = 4) +
      scale_fill_manual(values = c("orange", "dodgerblue")) +
      labs(x = "Pregnancy outcome",
           y = paste(g, "expression level"),
           fill = "Vitamin D status") +
      theme_classic(base_size = 14)

# Show the 3 plots (plots made inside a loop don't show up unless we print them)
    print(p1); print(p2); print(p3)



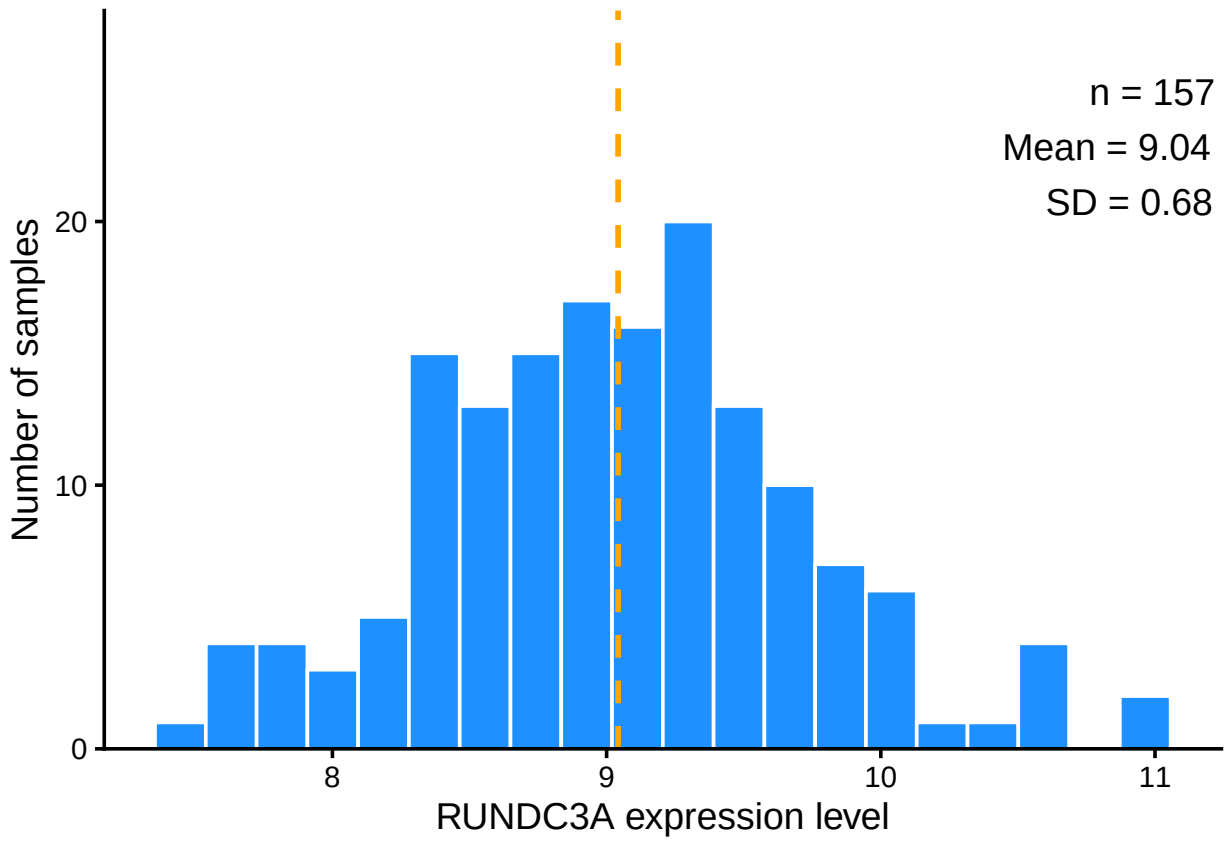

    #CUT OUT...UNECESSARY

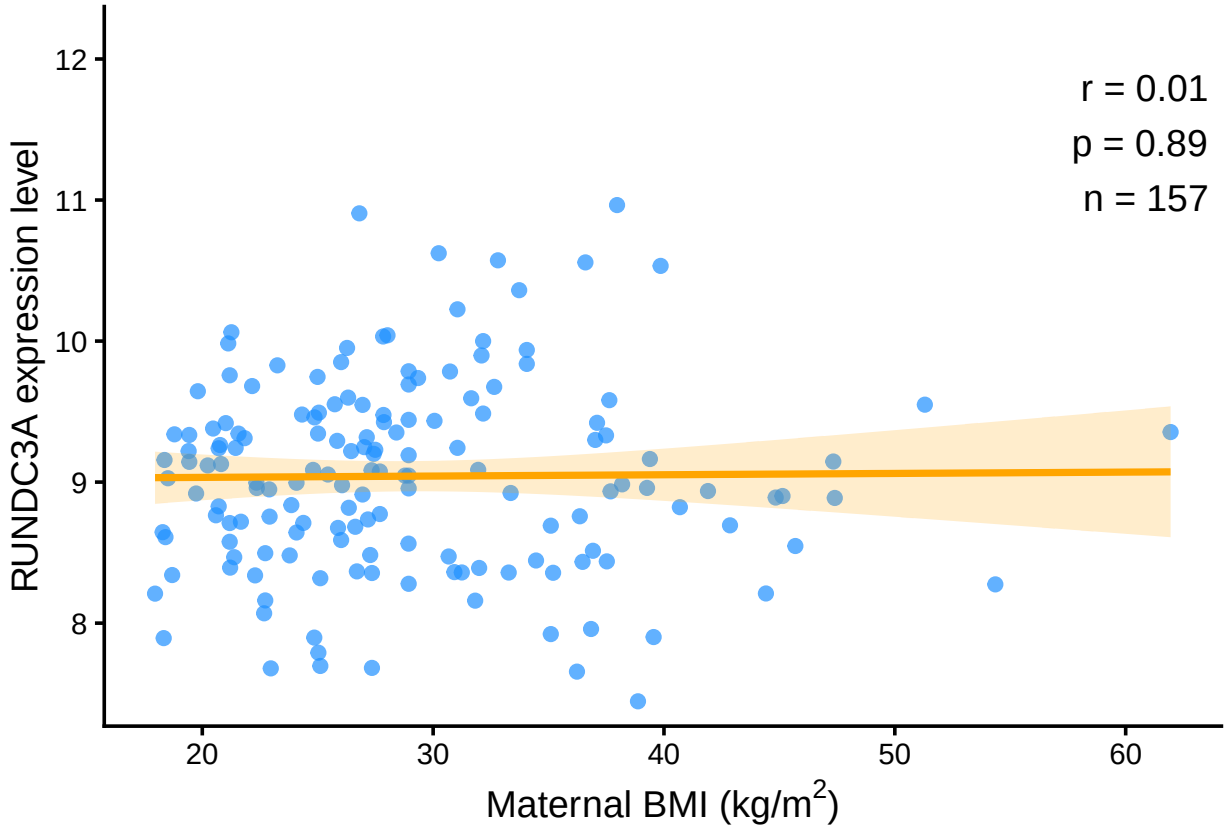
# Linear model with both categorical variables (used for the summary table)
fit <- lm(expression ~ preg_preeclampsia + vitD_status, data = d)
co  <- summary(fit)$coefficients

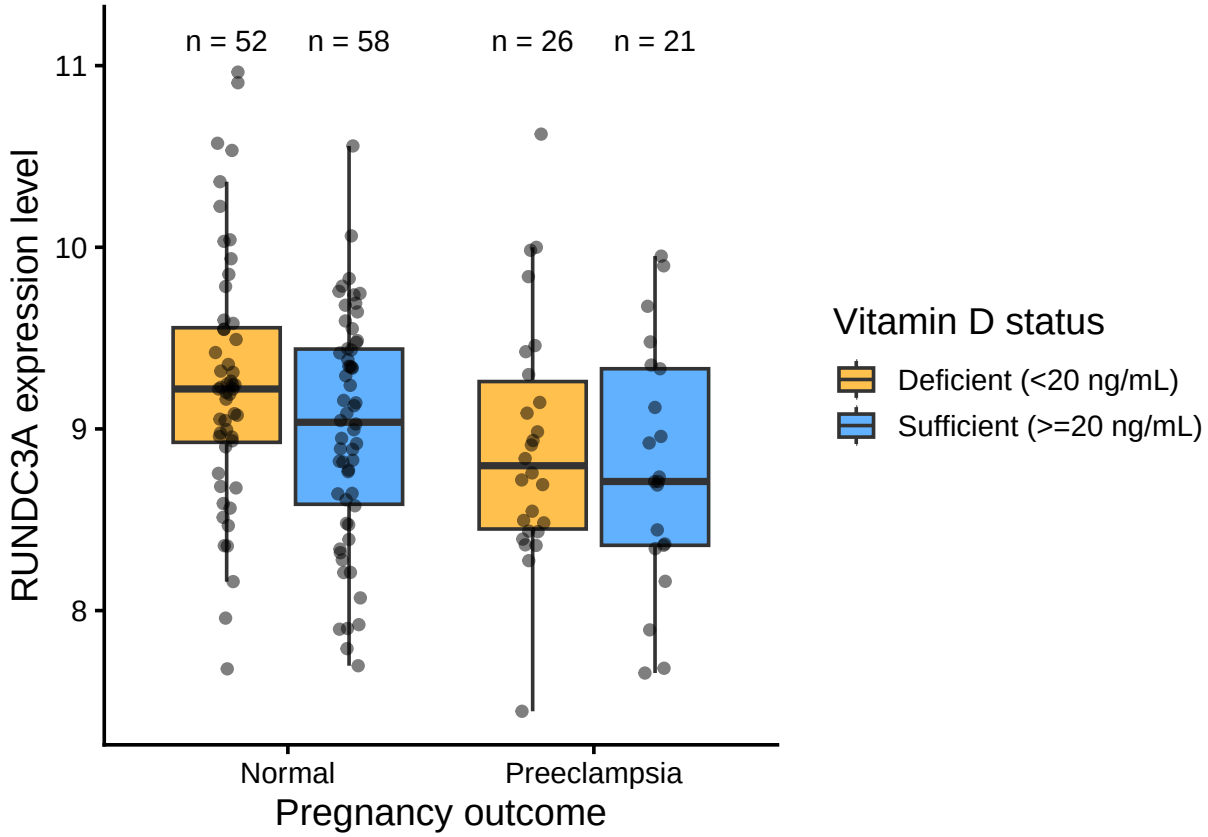
    # Save this gene's plots and its summary numbers
    plots[[g]] <- list(histogram = p1, scatter = p2, boxplot = p3)
    gene_stats[[g]] <- tibble(
      gene = g, mean = m, sd = s,
      r_BMI = unname(ct$estimate), p_BMI = ct$p.value,
      preeclampsia_effect = co[2, "Estimate"], p_preeclampsia = co[2, "Pr(>|t|)"],
      vitD_effect = co[3, "Estimate"], p_vitD = co[3, "Pr(>|t|)"])
  }

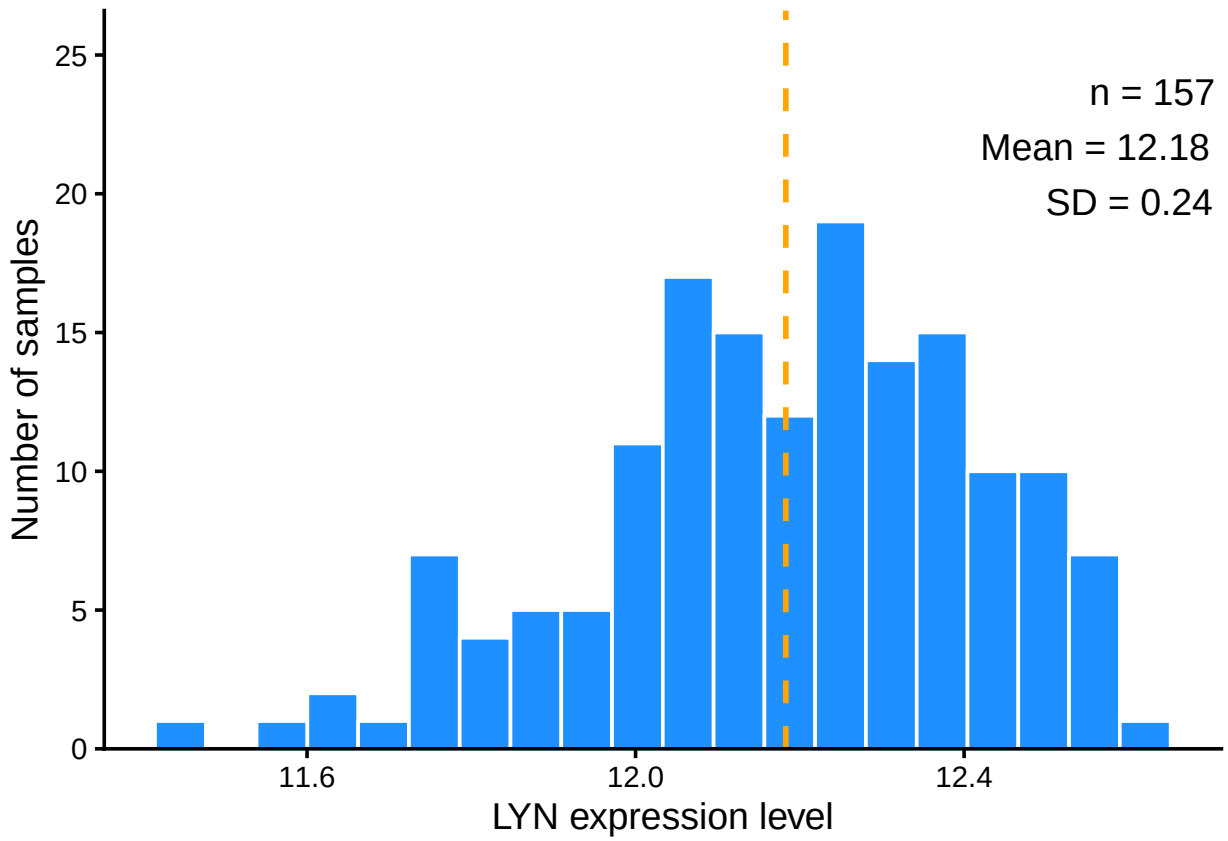
  invisible(list(plots = plots, stats = bind_rows(gene_stats)))
}
```

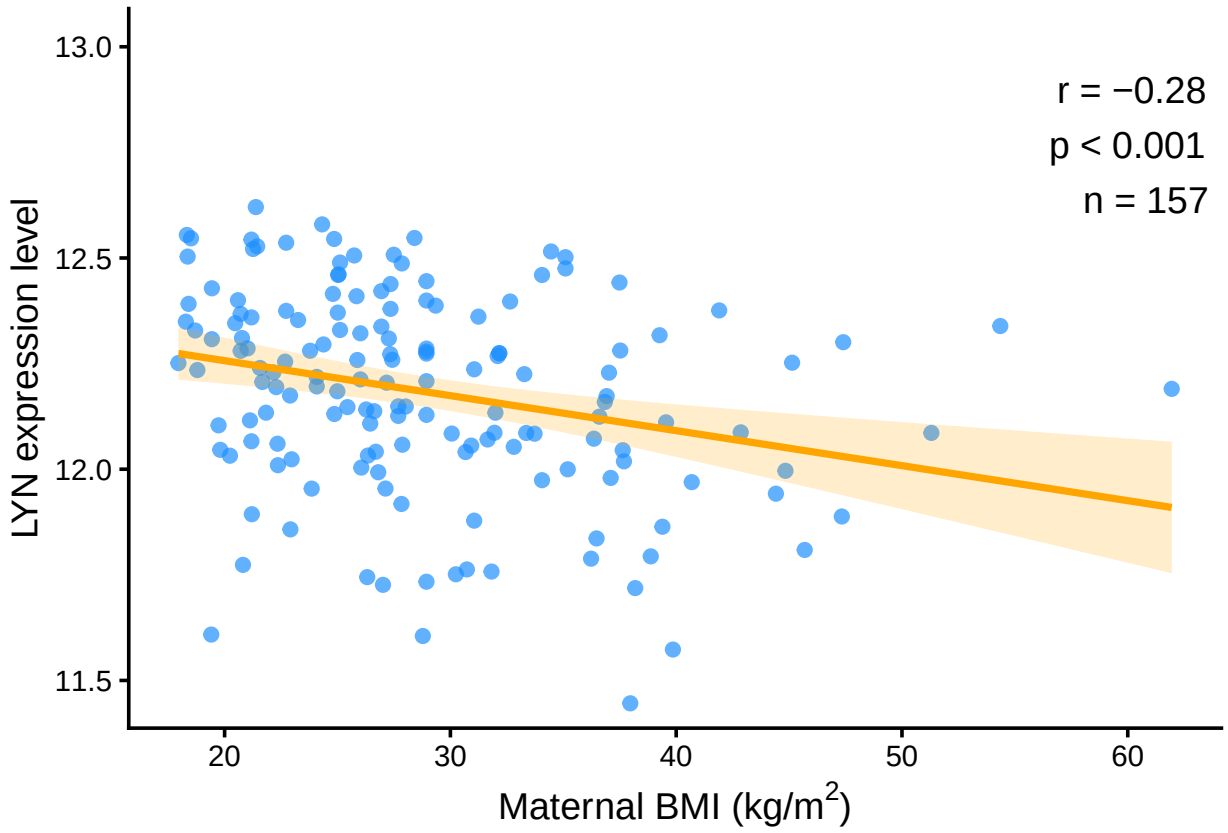
```
gene_results <- plot_gene_expression(dat_all, genes_of_interest)
```

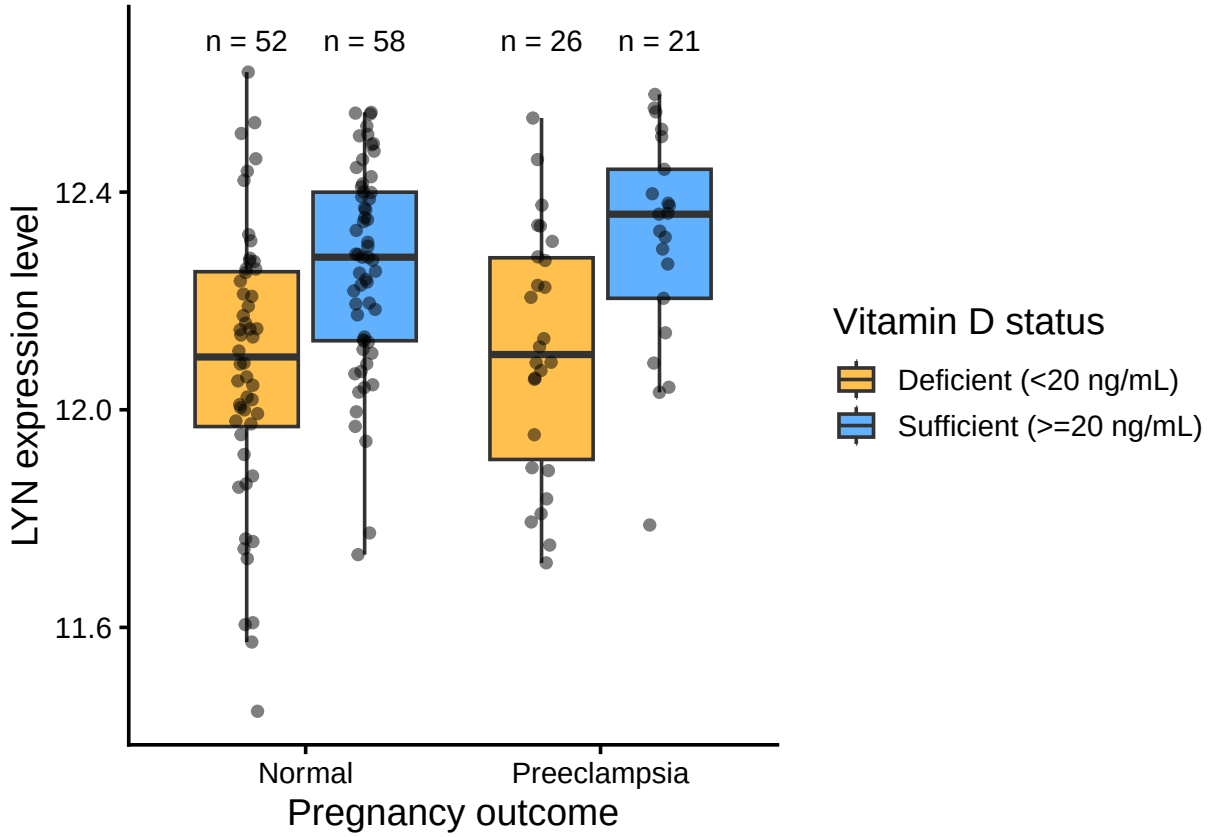

n = 157
Mean = 9.04
SD = 0.68
Number of samples
20
10
0
8
9
10
11
RUNDC3A expression level

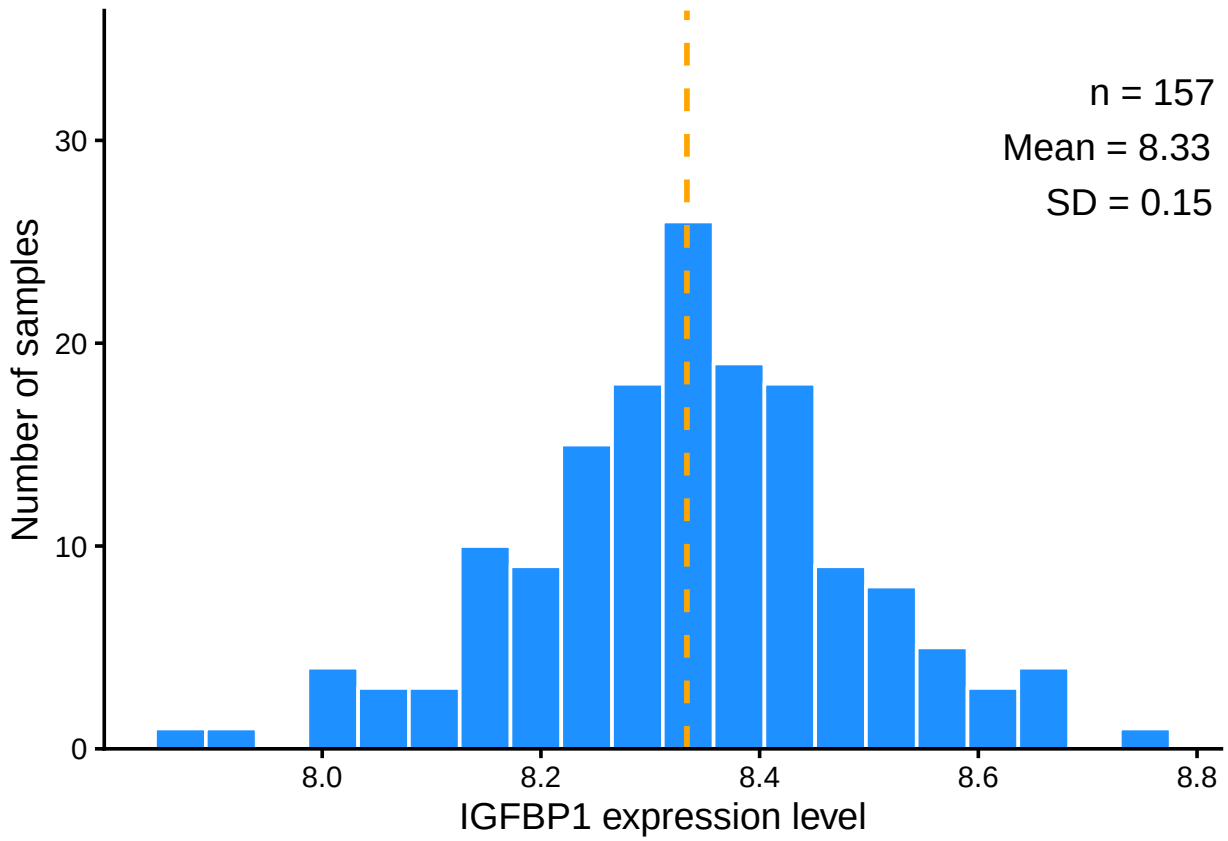

r = 0.01
p = 0.89
n = 157
RUNDC3A expression level
12
11
10
9
8
20
30
40
50
60
Maternal BMI (kg/m2)

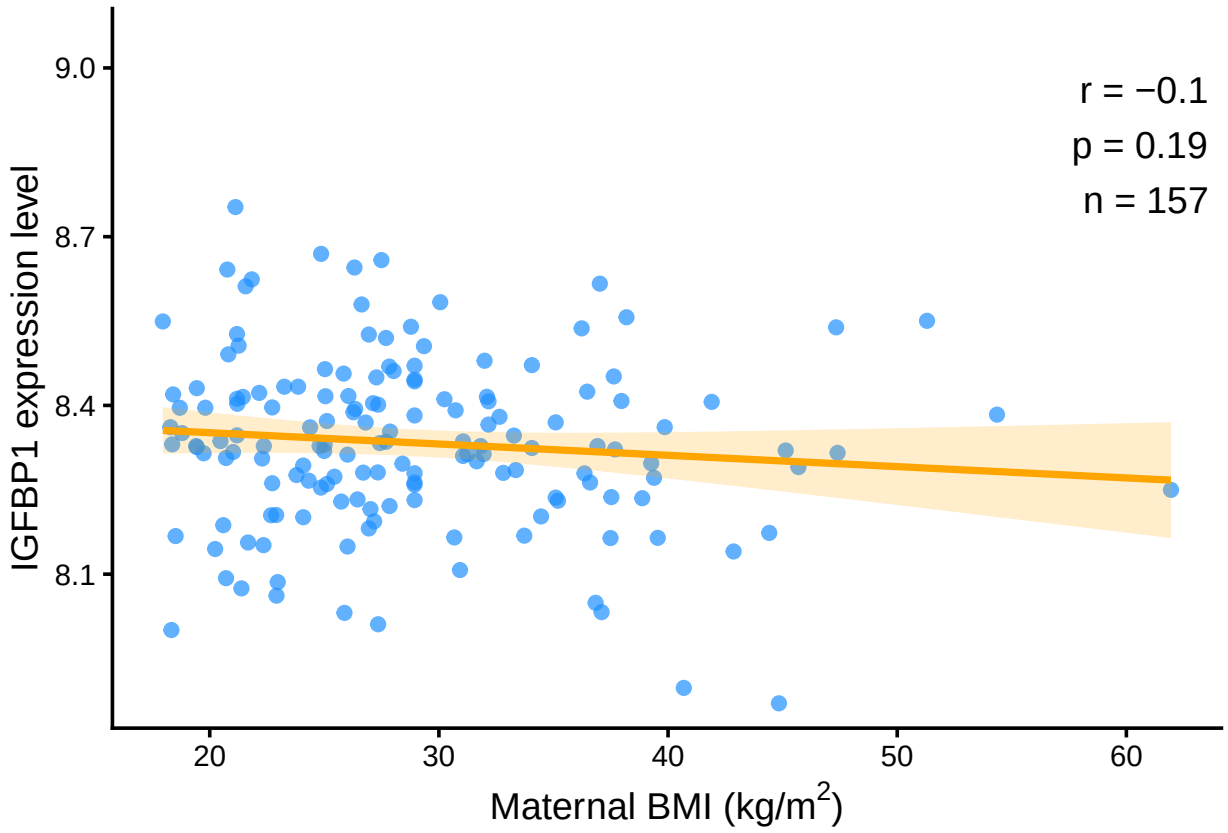

n = 52
n = 58
n = 26
n = 21
11
10
9
8
RUNDC3A expression level
Normal
Preeclampsia
Pregnancy outcome
Vitamin D status
Deficient (<20 ng/mL)
Sufficient (>=20 ng/mL)

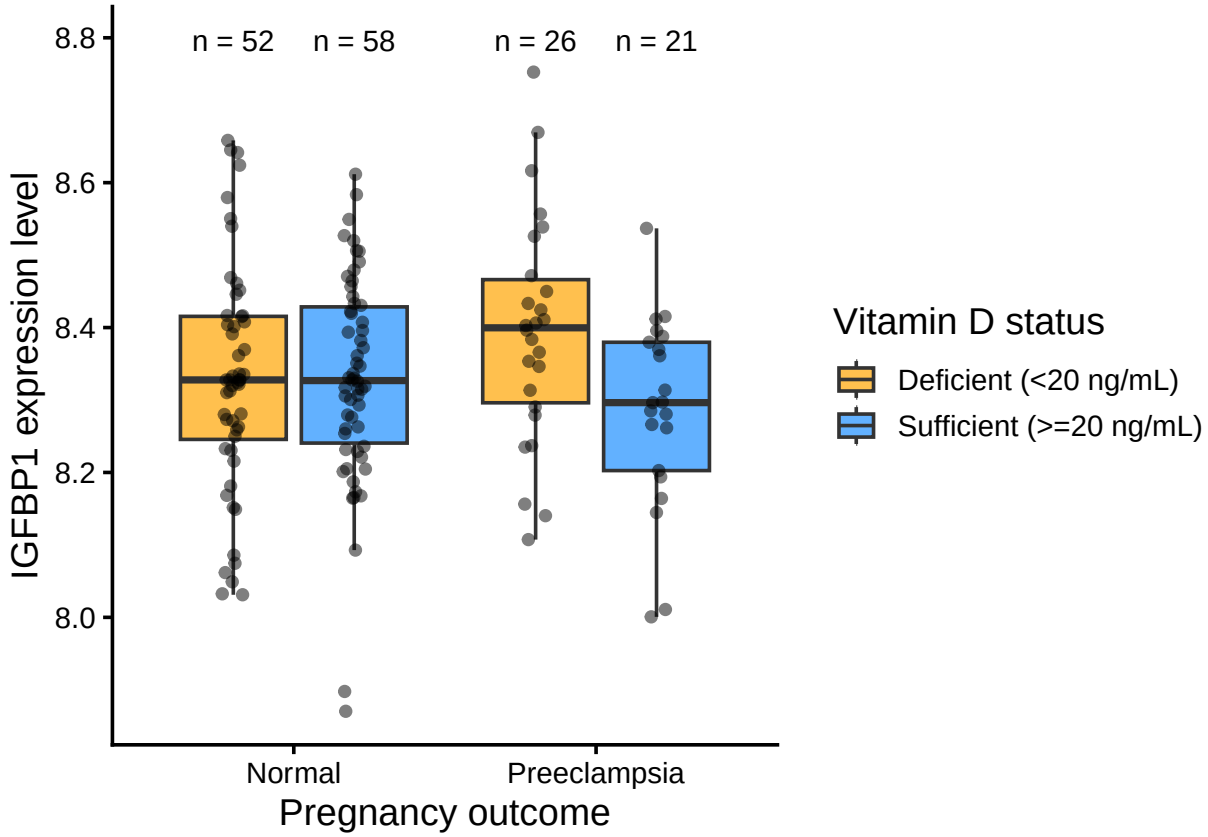

n = 157
Mean = 12.18
SD = 0.24
Number of samples
25
20
15
10
5
0
11.6
12.0
12.4
LYN expression level


r = −0.28
p < 0.001
n = 157
LYN expression level
13.0
12.5
12.0
11.5
20
30
40
50
60
Maternal BMI (kg/m2)


n = 52
n = 58
n = 26
n = 21
12.4
12.0
11.6
LYN expression level
Normal
Preeclampsia
Pregnancy outcome
Vitamin D status
Deficient (<20 ng/mL)
Sufficient (>=20 ng/mL)


n = 157
Mean = 8.33
SD = 0.15
Number of samples
0
10
20
30
8.0
8.2
8.4
8.6
8.8
IGFBP1 expression level


r = −0.1
p = 0.19
n = 157
IGFBP1 expression level
9.0
8.7
8.4
8.1
20
30
40
50
60
Maternal BMI (kg/m2)




```
# Additional packages
# pheatmap = clustered heatmap with tracking (annotation) bars
# knitr = kable() to turn a data frame into a LaTex table
# kableExtra = nicer LaTex tables (booktabs rules, grouped rows)
library(pheatmap)
library(knitr)
library(kableExtra)
```

```
##
## Attaching package: 'kableExtra'
```

```
## The following object is masked from 'package:dplyr':
##
##     group_rows
```

```
# Everything the LaTeX report needs gets saved into these two folders,
# so the report can \input / \includegraphics them directly.
dir.create("figures", showWarnings = FALSE)
dir.create("tables", showWarnings = FALSE)

# Final sample size check: ... does anyone have missing covariate values?
# If all zeros, nobody needs to be dropped and n stays at 157
colSums(is.na(GSE.covar))
```

```
##                    subjectID                        geoID
```

```
##                             0                             0
##                 mat_age_yrs           preg_preeclampsia
##                             0                             0
##         delivery_gestAge_wks sampleCollection_gestAge_wks
##                             0                             0
##             batch_Microarray                       mat_BMI
##                             0                             0
##                      mat_vitD                   mat_asthma
##                             0                             0
##                  vitD_status
##                             0
```

```
# Perfect!
nrow(GSE.covar)
```

```
## [1] 157
```

Step 7: Summary Statistics table (stratified by preeclampsia)

```
# The table needs 3 contiuous + 2 categorical covariates, stratified by a
# categorical covariate. We stratify by pregnancy outcome (categorical #1)
# and show:

# Continuous: maternal BMI (from our plots), maternal age, and gestational
# age at delivery (the 2 new continuous covariates)

# Categorical: vit D status (categorical #2, shown as n (%))
# Continuous variables are summarised as mean (SD) and compared with a
# Welch two-sample t-test. Vit D status is compared with a chi-squared test

# a Smalle helper: "mean (SD)" text for one numeric vector
mean_sd <- function(x) sprintf("%.1f (%.1f)", mean(x), sd(x))

# another small helper: format p-values the same way as the plots
fmt_p <- function(p) ifelse(p <0.001, "<0.001", sprintf("%.2f", p))

# The continuous covaraites and the labels we want printed in the table
cont_vars <- c(mat_BMI = "Maternal BMI (kg/m$^2$)",
               mat_age_yrs = "Maternal age (years)",
               delivery_gestAge_wks = "Gestational age at delivery (weeks)")

# One row per continuous covariate: overall, Normal, Preeclampsia, p-value
cont_rows <- map_dfr(names(cont_vars), function(v){
  x <- GSE.covar[[v]]
  grp <- GSE.covar$preg_preeclampsia
  tibble(
    Characteristic = cont_vars[[v]],
    Overall = mean_sd(x),
    Normal = mean_sd(x[grp == "Normal"]),
    Preeclampsia = mean_sd(x[grp == "Preeclampsia"]),
    P = fmt_p(t.test(x ~ grp)$p.value))

})
```

Table 1: Participant characteristics by pregnancy outcome

Characteristic	Overall (n = 157)	Normal (n = 110)	Preeclampsia (n = 47)	*p*-value
Maternal BMI (kg/m^2)	28.9 (7.9)	28.0 (7.7)	31.2 (8.0)	0.02
Maternal age (years)	26.4 (5.0)	26.6 (5.1)	25.8 (4.9)	0.33
Gestational age at delivery (weeks)	38.8 (2.2)	39.3 (1.0)	37.8 (3.5)	0.01
Vitamin D status, n (%)				0.45
Deficient (<20 ng/mL)	78 (49.7)	52 (47.3)	26 (55.3)	
Sufficient (≥20 ng/mL)	79 (50.3)	58 (52.7)	21 (44.7)	

```
# Vitamin D status: count and column percent within each pregnancy outcome
vitD_tab <- table(GSE.covar$vitD_status, GSE.covar$preg_preeclampsia)
n_pct <- function(n, total) sprintf("%d (%.1f)", n, 100 * n / total)

cat_rows <- tibble(
  Characteristic = c("Vitamin D status, n (\\%)",
                     "\\quad Deficient ($<$20 ng/mL)",
                     "\\quad Sufficient ($\\geq$20 ng/mL)"),
  Overall = c("", n_pct(rowSums(vitD_tab), nrow(GSE.covar))),
  Normal = c("", n_pct(vitD_tab[, "Normal"], sum(vitD_tab[, "Normal"]))),
  Preeclampsia = c("", n_pct(vitD_tab[, "Preeclampsia"], sum(vitD_tab[, "Preeclampsia"]))),
  P = c(fmt_p(chisq.test(vitD_tab)$p.value), "", ""))


# Stack the rows, then add the group sizes to the column names
table1 <- bind_rows(cont_rows, cat_rows)
n_grp <- table(GSE.covar$preg_preeclampsia)
colnames(table1) <- c("Characteristic",
                      sprintf("Overall (n = %d)", nrow(GSE.covar)),
                      sprintf("Normal (n = %d)", n_grp[["Normal"]]),
                      sprintf("Preeclampsia (n = %d)", n_grp[["Preeclampsia"]]),
                      "$p$-value")


# Write the LaTex version ( just the tabular; the report adds teh caption).
# escape = F so our LaTex symbols ($^2$, \quad, \%) are kept as LaTex.

table1_tex <- kable(table1, format = "latex", booktabs = T, linesep = "", escape = F, align = c("l", "c
writeLines(table1_tex, "tables/table1.tex")


# Also show it here in the knitted PDF
kable(table1, format = "latex", booktabs = T, escape = F, linesep = "",
      align = c("l", "c", "c", "c", "c"),
      caption = "Participant characteristics by pregnancy outcome") %>%
  kable_styling(latex_options = c("hold_positions", "scale_down"))
```

Step 8: Save the 9 plots from Submission 2 for the report

```
# gene_results (made in step 5) already holds every plot, so we just loop
# over it and save each one as a PDF.

for (g in names(gene_results$plots)){
  for (type in names(gene_results$plots[[g]])) {
    ggsave(filename = file.path("figures", paste0(g, "_", type, ".pdf")),
           plot = gene_results$plots[[g]][[type]],
           width = 6, height = 4.5)
  }
}


# The numbers behind the plots (correlations and model effects), used in
# the Results section of the report
gene_results$stats %>% mutate(across(where(is.numeric), ~signif(.x, 3)))
```

```
## # A tibble: 3 x 9
##   gene     mean    sd   r_BMI    p_BMI preeclampsia_effect p_preeclampsia
##   <chr>   <dbl> <dbl>   <dbl>    <dbl>               <dbl>          <dbl>
## 1 RUNDC3A  9.04 0.679  0.0108 0.893               -0.279          0.0172
## 2 LYN     12.2  0.237 -0.275  0.000484             0.0375         0.329
## 3 IGFBP1   8.33 0.152 -0.104  0.193                0.0145         0.586
## # i 2 more variables: vitD_effect <dbl>, p_vitD <dbl>
```

Step 9: Heatmap of the 10 most variable genes

```
# Which 10 genes? We pick the 10 genes whose expression varies the most across
# the 157 women (largest standard deviation). This choice does not use
# preeclampsia or Vit D at all, so the clustering is not "rigged"
# to seperate the groups

top10 <- GSE.GE %>%
  group_by(GENE) %>%
  summarise(sd = sd(expression)) %>%
  arrange(desc(sd)) %>%
  slice_head(n = 10)
top10
```

```
## # A tibble: 10 x 2
##    GENE        sd
##    <chr>    <dbl>
##  1 UTS2     1.03
##  2 RUNDC3A  0.679
##  3 TMEM176B 0.663
##  4 CEACAM6  0.650
##  5 CLIC2    0.646
##  6 SLC1A5   0.586
##  7 ZNF594   0.563
##  8 ZBTB41   0.562
##  9 CAPZA2   0.557
## 10 TGM2     0.524
```

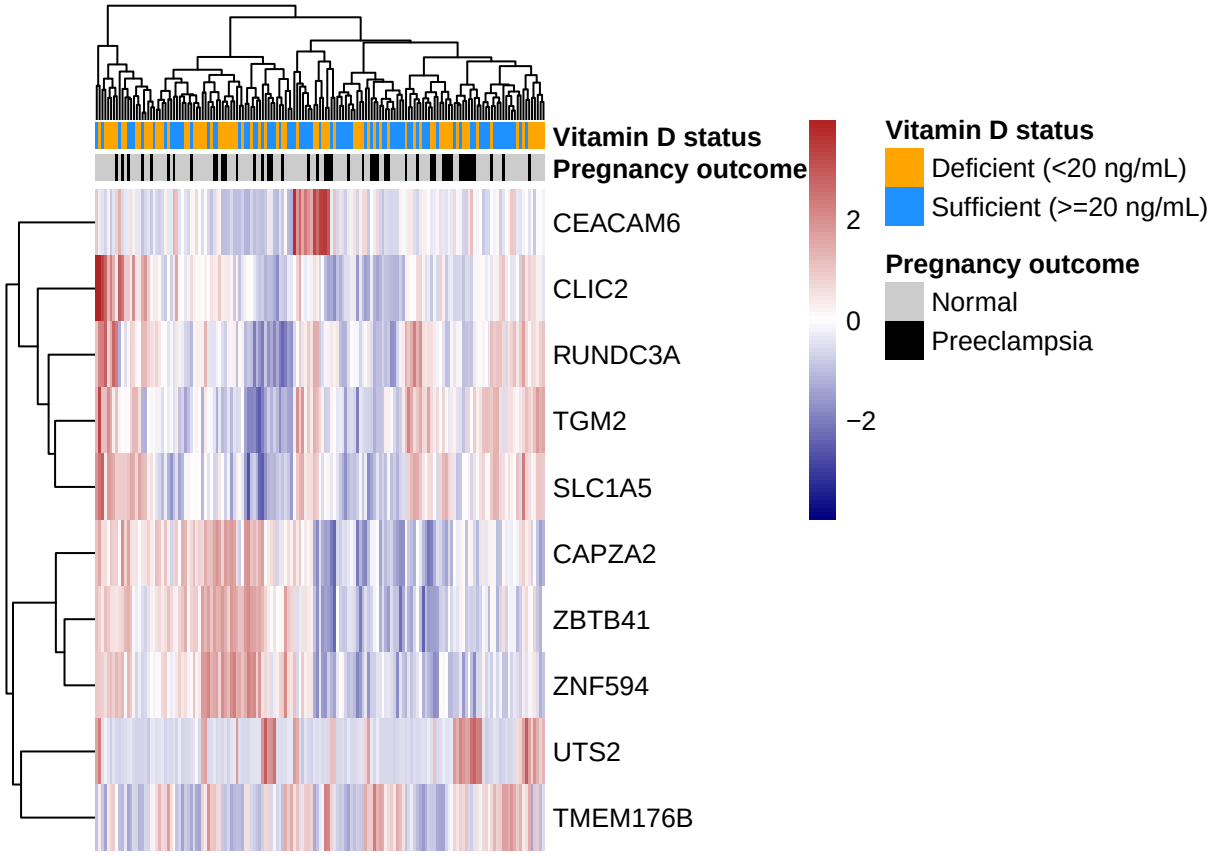
```r
# pheatmap nees a matrix: one row per gene, one column per women
heat_mat <- GSE.GE %>%
  filter(GENE %in% top10$GENE) %>%
  select(GENE, geoID, expression) %>%
  pivot_wider(names_from = geoID, values_from = expression) %>%
  column_to_rownames("GENE") %>%
  as.matrix()

# Tracking bars: one row per woman, named by geoID so pheatmap can line them
# up with the matrix columns
annot <- GSE.covar %>%
  select(geoID, preg_preeclampsia, vitD_status) %>%
  rename(`Pregnancy outcome` = preg_preeclampsia,
         `Vitamin D status` = vitD_status) %>%
  column_to_rownames("geoID")
annot <- annot[colnames(heat_mat), ]  # same order as the matrix colimns fyi

# Tracking bar colors (vit D matches teh boxplots: orange / blue)
annot_colors <- list(
  `Pregnancy outcome` = c(Normal = "grey80", Preeclampsia = "black"),
  `Vitamin D status` = c(`Deficient (<20 ng/mL)` = "orange",
                         `Sufficient (>=20 ng/mL)` = "dodgerblue"))


# scale = "row" turns each gene into z-scores, so genes with different
# average levels can share one color scale. Rows(genes) and columns(women)
# are both clustered with Euclidean distance + complete linkage.
heat_args <- list(mat = heat_mat,
                  scale ="row",
                  clustering_distance_rows = "euclidean",
                  clustering_distance_cols = "euclidean",
                  clustering_method = "complete",
                  annotation_col = annot,
                  annotation_colors = annot_colors,
                  color = colorRampPalette(c("navy", "white", "firebrick"))(100),
                  show_colnames = F,
                  treeheight_row = 30, treeheight_col = 40,
                  fontsize =10)

do.call(pheatmap, heat_args)
```




```
do.call(pheatmap, c(heat_args, list(filename = "figures/heatmap.pdf", width = 9, height = 5)))
```

```
# Quick check: are the two main column clusters linked to preeclampsia?
col_clusters <- cutree(do.call(pheatmap, c(heat_args, list(silent = TRUE)))$tree_col, k = 2)
table(cluster = col_clusters, outcome = annot[names(col_clusters), "Pregnancy outcome"])
```

```
##         outcome
## cluster Normal Preeclampsia
##       1     94           42
##       2     16            5
```

```
chisq.test(table(col_clusters, annot[names(col_clusters), "Pregnancy outcome"]))$p.value
```

```
## [1] 0.6871616
```

Step 10: Additional plot: forestplot of adjusted effects

```
# New plot type is ... A forest plot!(aka coefficient plot)
# Instead of looking at one covariate at a time, we fit ONE model per gene. with all three covariates:
#     expression (z-score) ~ preeclampsia + Vit D status + BMI (z-score)
# Expression and BMI are standardised, so every effect is in "SDs of gene
# expression", which lets us compare the 3 genes on the same axis.
```

```r
# Dot = estimates, bars = 95% CI, dashed line = no effect.

forest_df <- map_dfr(genes_of_interest, function(g){
  d <- dat_all %>%
    filter(GENE == g) %>%
    mutate(expr_z = as.numeric(scale(expression)),
           bmi_z = as.numeric(scale(mat_BMI)))
  fit <- lm(expr_z ~ preg_preeclampsia + vitD_status + bmi_z, data = d)
  ci <- confint(fit)
  tibble(gene = g,
         term = rownames(ci)[-1],
         estimate = coef(fit)[-1],
         lower = ci[-1,1],
         upper= ci[-1,2],
         p = summary(fit)$coefficients[-1, "Pr(>|t|)"])
}) %>%
  mutate(term = recode(term,
                       preg_preeclampsiaPreeclampsia = "Preeclampsia\n(vs. normal)",
                       `vitD_statusSufficient (>=20 ng/mL)` = "Vitamin D sufficient\n(vs. deficient)",
                       bmi_z = "Maternal BMI\n(per 1 SD)"),
                       term = factor(term, levels = c("Preeclampsia\n(vs. normal)",
                                                      "Vitamin D sufficient\n(vs. deficient)",
                                                      "Maternal BMI\n(per 1 SD)")),
                       gene = factor(gene, levels = rev(genes_of_interest)),
                       significant = ifelse(p < 0.05, "p < 0.05", "p >= 0.05"))

         forest_df %>%  mutate(across(where(is.numeric), ~ signif(.x, 3)))
```

```
## # A tibble: 9 x 7
##   gene    term                      estimate   lower    upper       p significant
##   <fct>   <fct>                        <dbl>   <dbl>    <dbl>   <dbl> <chr>
## 1 RUNDC3A "Preeclampsia\n(vs. nor~ -0.41    -0.754  -0.0671  1.95e-2 p < 0.05
## 2 RUNDC3A "Vitamin D sufficient\n~ -0.33    -0.652  -0.00687 4.54e-2 p < 0.05
## 3 RUNDC3A "Maternal BMI\n(per 1 S~ -0.00198 -0.166   0.162   9.81e-1 p >= 0.05
## 4 LYN     "Preeclampsia\n(vs. nor~  0.229   -0.0897  0.547   1.58e-1 p >= 0.05
## 5 LYN     "Vitamin D sufficient\n~  0.677    0.378   0.976   1.52e-5 p < 0.05
## 6 LYN     "Maternal BMI\n(per 1 S~ -0.197   -0.349  -0.0447  1.16e-2 p < 0.05
## 7 IGFBP1  "Preeclampsia\n(vs. nor~  0.153   -0.194   0.5     3.85e-1 p >= 0.05
## 8 IGFBP1  "Vitamin D sufficient\n~ -0.308   -0.634   0.0179  6.38e-2 p >= 0.05
## 9 IGFBP1  "Maternal BMI\n(per 1 S~ -0.162   -0.328   0.00413 5.59e-2 p >= 0.05
```

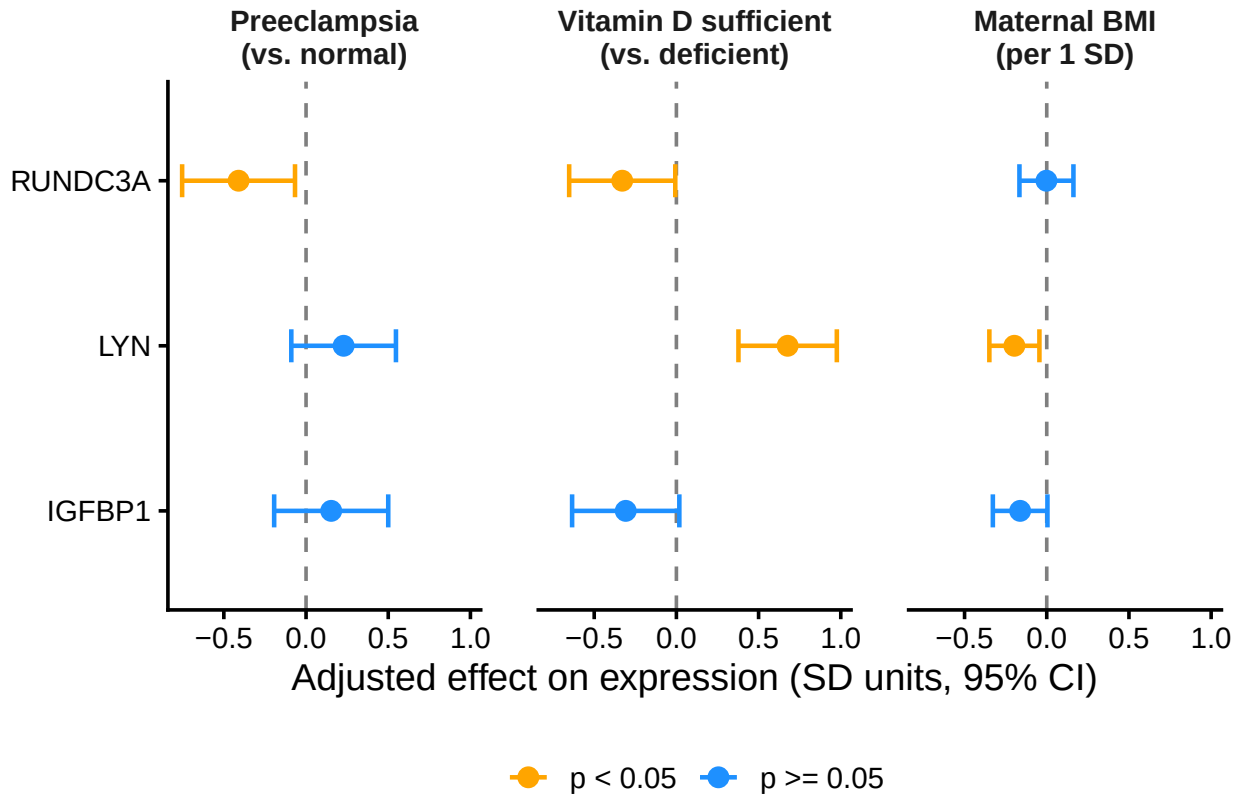
```r
p_forest <- ggplot(forest_df, aes(x = estimate, y = gene, color = significant)) +
  geom_vline(xintercept = 0, linetype = "dashed", color = "grey50") +
  geom_errorbarh(aes(xmin = lower, xmax = upper), height = 0.2, linewidth = 0.8) +
  geom_point(size = 3) +
  facet_wrap(~ term, nrow = 1) +
  scale_color_manual(values = c("p < 0.05" = "orange", "p >= 0.05" = "dodgerblue")) +
  labs(x = "Adjusted effect on expression (SD units, 95% CI)",
       y = NULL, color = NULL) +
  theme_classic(base_size = 14) +
  theme(legend.position = "bottom",
        strip.background = element_blank(),
        strip.text = element_text(face = "bold"),
```

```
        panel.spacing = unit(1.5, "lines"))
```

```
## Warning: `geom_errorbarh()` was deprecated in ggplot2 4.0.0.
## i Please use the `orientation` argument of `geom_errorbar()` instead.
## This warning is displayed once per session.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

```
p_forest
```

```
## `height` was translated to `width`.
```




```
ggsave("figures/forest.pdf", p_forest, width = 9, height = 4)
```

```
## `height` was translated to `width`.
```

```
# R and package verisons to cite in the actual report
R.version.string
```

```
## [1] "R version 4.6.1 (2026-06-24)"
```

```r
sapply(c("tidyverse", "ggplot2", "dplyr", "tidyr", "readr", "stringr",
"purrr", "tibble", "pheatmap", "knitr", "kableExtra", "rmarkdown"),
function(p) as.character(packageVersion(p)))
```

```
##  tidyverse    ggplot2      dplyr      tidyr      readr    stringr      purrr
##    "2.0.0"    "4.0.3"    "1.2.1"    "1.3.2"    "2.2.0"    "1.6.0"    "1.2.2"
##     tibble   pheatmap      knitr kableExtra  rmarkdown
##    "3.3.1"   "1.0.13"     "1.52"    "1.4.1"     "2.32"
```

```r
sessionInfo()
```

```
## R version 4.6.1 (2026-06-24)
## Platform: aarch64-apple-darwin23
## Running under: macOS Tahoe 26.5.2
##
## Matrix products: default
## BLAS:   /Library/Frameworks/R.framework/Versions/4.6/Resources/lib/libRblas.0.dylib
## LAPACK: /Library/Frameworks/R.framework/Versions/4.6/Resources/lib/libRlapack.dylib;  LAPACK version
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## time zone: America/New_York
## tzcode source: internal
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
##  [1] kableExtra_1.4.1 knitr_1.52       pheatmap_1.0.13  lubridate_1.9.5
##  [5] forcats_1.0.1    stringr_1.6.0    dplyr_1.2.1      purrr_1.2.2
##  [9] readr_2.2.0      tidyr_1.3.2      tibble_3.3.1     ggplot2_4.0.3
## [13] tidyverse_2.0.0
##
## loaded via a namespace (and not attached):
##  [1] utf8_1.2.6          generics_0.1.4      xml2_1.6.0          lattice_0.22-9
##  [5] stringi_1.8.9       hms_1.1.4           digest_0.6.39       magrittr_2.0.5
##  [9] evaluate_1.0.5      grid_4.6.1          timechange_0.4.0    RColorBrewer_1.1-3
## [13] fastmap_1.2.0       Matrix_1.7-5        mgcv_1.9-4          viridisLite_0.4.3
## [17] scales_1.4.0        textshaping_1.0.5   cli_3.6.6           rlang_1.3.0
## [21] crayon_1.5.3        bit64_4.8.6         splines_4.6.1       withr_3.0.3
## [25] yaml_2.3.12         otel_0.2.0          tools_4.6.1         parallel_4.6.1
## [29] tzdb_0.5.0          vctrs_0.7.3         R6_2.6.1            lifecycle_1.0.5
## [33] bit_4.6.0           vroom_1.7.1         ragg_1.5.2          pkgconfig_2.0.3
## [37] pillar_1.11.1       gtable_0.3.6        glue_1.8.1          systemfonts_1.3.2
## [41] xfun_0.61           tidyselect_1.2.1    rstudioapi_0.19.0   farver_2.1.2
## [45] htmltools_0.5.9     nlme_3.1-169        svglite_2.2.2       rmarkdown_2.32
## [49] labeling_0.4.3      compiler_4.6.1      S7_0.2.2
```

ADDITIONAL RESOURCES USED:

- Google & Gemini: to look up commands we learned in class, and extra commands for more advanced arguments.

- Claude (sonnet, medium): to check my work for errors, and help me 'dumb down'/understand specific characteristics in the data and rstudio itself.